

UAJS SMART CAMPUS

Documento técnico — Entregable 1: Arquitectura y Prototipo Funcional

Corporación Universitaria Antonio José de Sucre — Sistemas Distribuidos

1. Objetivo

Diseñar y desarrollar la primera versión funcional y arquitectónica de UAJS Smart Campus mediante React, backend base, API Gateway y estructura inicial de microservicios, aplicando componentes reutilizables, React Router, hooks, lógica funcional, APIs REST, control de versiones y trazabilidad

2. Alcance

Frontend React funcional, backend base organizado, API Gateway operativo, seis microservicios estructurados, endpoints REST iniciales e integraciones funcionales con Solicitudes y Reservas. La persistencia completa y comunicación avanzada se profundizarán posteriormente

3. Actores

Estudiante, Docente, Personal administrativo, Administrador y Personal de mantenimiento

4. Arquitectura

USUARIO → FRONTEND REACT → API GATEWAY → USUARIOS / SOLICITUDES / RESERVAS / RECURSOS / EVENTOS / NOTIFICACIONES. El frontend consume el Gateway en 8080 y la comunicación utiliza HTTP/JSON

5. Frontend

Landing, Login, Dashboard, Servicios, Recursos, Reservas, Solicitudes, Detalle, Eventos, Notificaciones, Perfil y PQRS. Se utilizan JSX, useState, useEffect, useMemo y el custom hook useCampusData

6. Lógica

Solicitudes: creación, consulta, búsqueda y detalle. Reservas: creación, consulta, búsqueda, detalle, cambio de estado y eliminación. El backend de Reservas valida conflictos de horario

7. Integraciones

React → API Gateway → Requests Service y React → API Gateway → Reservations Service

8. CSS y BEM

Las vistas complementarias usan clases BEM con prefijo sc-, por ejemplo sc-landing__header, sc-service-card__icon y sc-profile__grid

9. GitHub y ClickUp

La guía solicita ramas, commits significativos, Pull Requests, tareas con responsables y evidencias, y recomienda relacionar IDs de ClickUp con ramas, commits o PRs

10. Estado

Frontend, Dashboard, Recursos, Solicitudes, Reservas, API Gateway e integraciones principales están implementados. El cierre definitivo requiere completar y verificar los health checks de los seis microservicios y las evidencias finales

11. Servicios

Servicio	Puerto	Responsabilidad
Usuarios	8081	Autenticación y usuarios
Solicitudes	8082	Solicitudes
Reservas	8083	Reservas
Recursos	8084	Recursos del campus
Notificaciones	8085	Notificaciones
Eventos	8086	Eventos

12. Evidencias a anexar

Capturas del Dashboard, Recursos, Solicitudes, Reservas, Eventos, Notificaciones, Perfil, PQRS, rutas React Router, consola del Gateway, respuestas JSON, GitHub, Pull Requests y ClickUp